

Program 1:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main(){
int src, dest;
char buffer[1024];
int bytes;
src = open("input.txt", O_RDONLY);
if(src < 0){
printf("Cannot open source file\n");
return 1;
}
dest = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
while((bytes = read(src, buffer, sizeof(buffer))) > 0){
write(dest, buffer, bytes);
}
close(src);
close(dest);
printf("File copied successfully.\n");
return 0;
}
```

Output:

```
crazyguy64@CrazyGuy64:~/OS_WEEK_2$ nano input.txt
crazyguy64@CrazyGuy64:~/OS_WEEK_2$ nano output.txt
crazyguy64@CrazyGuy64:~/OS_WEEK_2$
crazyguy64@CrazyGuy64:~/OS_WEEK_2$ gcc program1.c -o program1
crazyguy64@CrazyGuy64:~/OS_WEEK_2$ ./program1
File copied successfully.
crazyguy64@CrazyGuy64:~/OS_WEEK_2$ |
```

Program 2:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(){
char cmd[100];

printf("Enter Linux command: ");
scanf("%s", cmd);
pid_t pid = fork();

if(pid == 0){
printf("Child PID : %d\n", getpid());
execlp(cmd, cmd, NULL);
perror("Execution Failed");
}
else if(pid > 0){
printf("Parent PID : %d\n", getpid());
wait(NULL);
printf("Child process completed. \n");
}else{
printf("Fork Failed\n");
}
return 0;
}
```

Output:

```
crazyguy64@CrazyGuy64:~/OS_WEEK_2$ nano program2.c
crazyguy64@CrazyGuy64:~/OS_WEEK_2$ gcc program2.c -o program2
crazyguy64@CrazyGuy64:~/OS_WEEK_2$ ./program2
Enter Linux command: pwd
Parent PID : 2470
Child PID : 2475
/home/crazyguy64/OS_WEEK_2
Child process completed.
```